

PDO – PHP Data Objects

Théorie et concepts fondamentaux

Prérequis : PHP de base | SQL de base | Environnement WSL

Table des matières

1 – Qu'est-ce que PDO ?	3
1.1 Le problème des anciennes fonctions PHP	3
1.2 Définitions de PDO	3
1.3 Les drivers PDO	3
1.4 Les 4 avantages de PDO.....	3
2 – Connexion et DSN	4
2.1 Le DSN (Data Source Name)	4
2.2 Les options PDO indispensables	4
2.3 Les exceptions et try/catch	5
3 – SQL et base de données	5
3.1 Les types de données MySQL.....	5
3.2 Les contraintes SQL	6
4 – Lire des données	6
4.1 La méthode query() et l'objet PDOStatement	6
4.2 Comparatif fetch() vs fetchAll()	7
5 – Les modes de récupération (fetch modes)	7
5.1 FETCH_NUM – Tableau indexé (indices numériques)	7
5.2 FETCH_ASSOC – Tableau associatif (RECOMMANDE)	8
5.3 FETCH_OBJ – Objet anonyme stdClass.....	8
6 – Sécurité : les injections SQL	8
6.1 Définition et mécanisme	8
6.2 Les 3 types d'attaque.....	9
6.3 Première protection : PDO::quote()	9
7 – Passage de paramètres HTTP.....	10
7.1 Pourquoi passer des paramètres ?.....	10
7.2 Comparatif des 4 méthodes.....	10
7.3 GET et POST en détail.....	10
7.4 SESSION et COOKIE : persistance entre pages	10
8 – Requetés préparées.....	11
8.1 Pourquoi les requêtes préparées ?.....	11
8.2 Les placeholders : ? et :nom.....	11
Syntaxe 1 : Placeholders positionnels (?)	11
Syntaxe 2 : Placeholders nommés (:nom).....	12
8.3 bindParam() vs bindValue()	12
bindParam() : lie par REFERENCE (adresse mémoire)	12
bindValue() : lie par VALEUR (copie immédiate).....	12
Synthèse – Les 8 concepts fondamentaux de PDO	13

1 – Qu'est-ce que PDO ?

1.1 Le problème des anciennes fonctions PHP

Avant PHP 7, il existait des fonctions différentes pour chaque type de base de données. Ces fonctions étaient incompatibles entre elles :

- **Pas de portabilité** : changer de BDD imposait de réécrire tout le code.
- **Pas de requêtes préparées** : les fonctions `mysql_*` ne protégeaient pas contre les injections SQL.
- **Gestion des erreurs silencieuse** : les erreurs passaient sans être signalées.
- **Supprimées en PHP 7** : `mysql_connect()`, `mysql_query()` n'existent plus.

1.2 Définitions de PDO

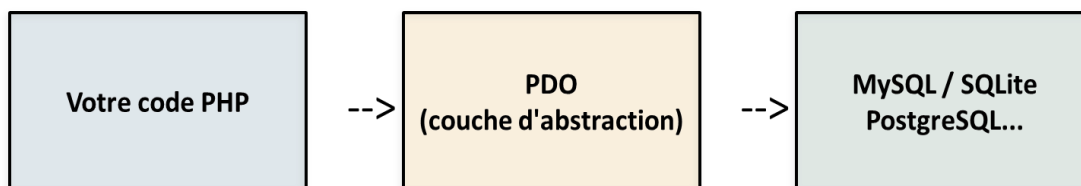
Définition

PDO (PHP Data Objects) est une extension PHP qui fournit une couche d'ABSTRACTION pour l'accès aux bases de données.

Abstraction = votre code PHP ne sait pas (et n'a pas besoin de savoir) s'il parle à MySQL, SQLite ou PostgreSQL. Il parle toujours à PDO avec la même syntaxe.

Constructeur : `new PDO(dsn, utilisateur, mot_de_passe, options)`

Le schéma d'abstraction :



1.3 Les drivers PDO

Un driver PDO est un module PHP spécifique à chaque type de base de données. Il traduit les commandes PDO génériques en commandes spécifiques au serveur cible.

Driver	Base de données et usage
<code>pdo_mysql</code>	MySQL / MariaDB – le plus utilisé en développement web
<code>pdo_sqlite</code>	SQLite – fichier local, idéal pour les tests et prototypes
<code>pdo_pgsql</code>	PostgreSQL – bases de données avancées
<code>pdo_sqlsrv</code>	SQL Server – environnements Microsoft

1.4 Les 4 avantages de PDO

- **1 seule API** : une syntaxe identique pour tous les types de bases de données.
- **Portabilité** : changer de BDD = changer seulement la chaîne de connexion (DSN : Data Source Name).
- **Sécurité intégrée** : requêtes préparées protégées contre les injections SQL nativement.
- **Moderne** : exceptions PHP, modes de récupération configurables, gestion des erreurs claire.

2 – Connexion et DSN

2.1 Le DSN (Data Source Name)

Définition

Le DSN (Data Source Name) est une chaîne de caractères qui identifie de façon unique une source de données.

Il indique à PDO : quel driver utiliser, l'adresse du serveur, le nom de la base, l'encodage.

Format général : 'driver:cle=valeur;cle=valeur;...'

Exemple de DSN MySQL

```
$dsn = 'mysql:host=localhost;dbname=ws_pdo;charset=utf8mb4';  
//      -----  
//      (1)      (2)      (3)      (4)  
  
// (1) mysql   = le driver (type de base de données)  
// (2) host    = adresse du serveur MySQL (localhost sur WSL)  
// (3) dbname  = nom exact de la base de données  
// (4) charset = encodage (utf8mb4 supporte les accents et emojis)
```

Partie du DSN	Rôle
mysql	Driver à utiliser. Autres : sqlite, pgsql, sqlsrv...
host=localhost	Adresse du serveur MySQL. Sur WSL : toujours localhost.
dbname=ws_pdo	Nom exact de la base de données créée dans phpMyAdmin.
charset=utf8mb4	Encodage des caractères. utf8mb4 supporte les accents.

2.2 Les options PDO indispensables

PDO peut être configuré via un tableau d'options passé en 4ème argument du constructeur. Trois options sont indispensables dans tout projet :

Les 3 options recommandées

```
$options = [  
    PDO::ATTR_ERRMODE          => PDO::ERRMODE_EXCEPTION,  
    // Sans : les erreurs SQL retournent FALSE silencieusement  
    // Avec : PDO lève une PDOException -> visible et gérée  
  
    PDO::ATTR_DEFAULT_FETCH_MODE => PDO::FETCH_ASSOC,  
    // fetch() retourne un tableau associatif par défaut  
    // $row['pseudo'] au lieu de $row[0] (plus lisible)  
  
    PDO::ATTR_EMULATE_PREPARES => false,  
    // false = vraies requêtes préparées MySQL (plus sécurisé)  
];
```

Option	Effet
ERRMODE_EXCEPTION	Les erreurs SQL lèvent une PDOException au lieu de retourner FALSE silencieusement.
FETCH_ASSOC	fetch() retourne \$row['nom'] au lieu de \$row[0] et \$row['nom'] en double.
EMULATE_PREPARES=false	Force les vraies requêtes préparées cote MySQL. Sécurité maximale.

2.3 Les exceptions et try/catch

Qu'est-ce qu'une exception PHP ?

Une exception est un objet PHP qui représente une erreur.

Quand PDO rencontre un problème (connexion impossible, SQL invalide...), il crée une PDOException et l'envoie au programme.

Si personne ne l'attrape -> le programme s'arrête de façon incontrôlée.

Si elle est attrapée (catch) -> on réagit proprement et on affiche un message utile.

Structure try/catch avec PDO

```
try {
    $pdo = new PDO($dsn, $user, $password, $options);
    // Si connexion impossible -> PDOException levée -> goto catch
    echo 'Connexion réussie !';
} catch (PDOException $e) {
    // $e->getMessage() contient le message d'erreur MySQL
    echo 'Erreur : ' . $e->getMessage();
    exit(1); // Stopper : inutile de continuer
}
```

Les erreurs de connexion les plus fréquentes :

Erreur MySQL	Cause probable
SQLSTATE[HY000][1045] Access denied	Mauvais nom d'utilisateur ou mot de passe
SQLSTATE[HY000][1049] Unknown db	La base de données n'existe pas encore
SQLSTATE[HY000][2002] Refused	Le serveur MySQL n'est pas démarré

3 – SQL et base de données

3.1 Les types de données MySQL

Chaque colonne d'une table MySQL a un type qui définit quelles données elle peut stocker :

Type MySQL	Description et usage
INT	Entier. Identifiants (id), âges, compteurs. Ex : id INT NOT NULL
VARCHAR(n)	Texte de longueur variable, max n caractères. Ex : pseudo VARCHAR(30)

BOOLEAN	Vrai/Faux. En MySQL = TINYINT(1). 0=false, 1=true. Ex : statutAdmin BOOLEAN
TEXT	Texte long sans limite fixe. Usage : articles, descriptions.
DATETIME	Date et heure. Usage : date de création, modification.
DECIMAL(p, s)	Nombre décimal de précision exacte. Usage : prix. Ex : prix DECIMAL(10,2)

3.2 Les contraintes SQL

Les contraintes sont des règles appliquées aux colonnes pour garantir l'intégrité des données :

1. NOT NULL

Interdit la valeur NULL. La valeur est obligatoire lors de l'insertion.

Exemple : pseudo VARCHAR(30) NOT NULL -> Impossible d'insérer une ligne sans pseudo.

2. AUTO_INCREMENT

MySQL génère automatiquement une valeur entière croissante pour chaque ligne.

Exemple : id INT NOT NULL AUTO_INCREMENT -> Ligne 1 : id=1, Ligne 2 : id=2... (jamais à fournir)

3. DEFAULT valeur

Définit la valeur par défaut si la colonne n'est pas renseignée lors de l'insertion.

Exemple : statutAdmin BOOLEAN DEFAULT 0 -> Si non précise, vaut 0 (non-admin).

4. PRIMARY KEY

Désigne la colonne comme identifiant unique de la table. Chaque valeur doit être unique.

Exemple : PRIMARY KEY (id) -> Impossible d'avoir deux lignes avec le même id.

4 – Lire des données

4.1 La méthode query() et l'objet PDOStatement

PDO::query(string \$sql) : PDOStatement

query() envoie une requête SQL directement à MySQL et retourne un objet PDOStatement.

Cet objet représente un curseur pointé sur les résultats.

Il permet de lire les résultats ligne par ligne avec fetch() ou en une fois avec fetchAll().

IMPORTANT : query() ne doit être utilisé QUE pour des requêtes sans variable externe.

Dès qu'une variable est impliquée -> utiliser prepare() (voir 8).

Qu'est-ce qu'un PDOStatement ?

Un PDOStatement est un objet qui représente une requête préparée ou exécutée.

Il contient un curseur interne qui pointe sur la ligne courante des résultats.

Chaque appel à fetch() fait avancer ce curseur d'une position.

Quand le curseur arrive à la fin, fetch() retourne FALSE.

Attention – Comportement du curseur

Ne jamais appeler `fetch()` plusieurs fois de suite si vous voulez relire depuis le début.

Il faut exécuter `query()` à nouveau pour remettre le curseur au début.

Ou bien commenter les blocs précédents avant d'en tester un nouveau.

4.2 Comparatif `fetch()` vs `fetchAll()`

Critère	<code>fetch()</code>	<code>fetchAll()</code>
Retourne	1 ligne (tableau/objet)	Toutes les lignes (tableau 2D)
Avance le curseur	Oui, d'une position	Vide le resultset entier
Quand vide	Retourne FALSE	Retourne [] (tableau vide)
Usage typique	Boucle while, 1 résultat attendu	Afficher une liste complète
Mémoire	Economique (ligne par ligne)	Charge tout en mémoire

`fetchColumn()` – troisième méthode utile

`fetchColumn()` retourne une seule valeur scalaire (une cellule unique).

```
$count = $pdo->query('SELECT COUNT(*) FROM utilisateurs')->fetchColumn();
```

Retourne directement 5 (le nombre), pas un tableau.

5 – Les modes de récupération (fetch modes)

PDO peut retourner les données sous trois formes différentes selon le mode passe à `fetch()`. La ligne MySQL source est identique pour les 3 exemples ci-dessous :

Ligne MySQL source utilisée dans les exemples

<code>id=1 pseudo='Gandalf' motDePasse='Maia' statutAdmin=1</code>

5.1 `FETCH_NUM` – Tableau indexé (indices numériques)

PHP – `FETCH_NUM`

```
$ligne = $stmt->fetch(PDO::FETCH_NUM);  
// Resultat :  
$ligne[0] = 1           // id           (1ere colonne = indice 0)  
$ligne[1] = 'Gandalf'   // pseudo  
$ligne[2] = 'Maia'      // motDePasse  
$ligne[3] = 1           // statutAdmin  
  
echo $ligne[3]; // -> 1
```

- **(+)** Simple à écrire.
- **(-)** Fragile : si l'ordre des colonnes change dans `SELECT`, tous les indices changent. Peu lisible.

5.2 FETCH_ASSOC – Tableau associatif (RECOMMANDE)

PHP – FETCH_ASSOC

```
$ligne = $stmt->fetch(PDO::FETCH_ASSOC);  
// Resultat :  
$ligne['id']           = 1  
$ligne['pseudo']       = 'Gandalf'  
$ligne['motDePasse']   = 'Maia'  
$ligne['statutAdmin'] = 1  
  
echo $ligne['statutAdmin']; // -> 1 (beaucoup plus lisible !)
```

- **(+)** Lisible et stable : indépendant de l'ordre des colonnes dans SELECT.
- **(+)** Recommande dans tous les projets.
- **(-)** Aucun inconvénient majeur.

5.3 FETCH_OBJ – Objet anonyme stdClass

PHP – FETCH_OBJ

```
$ligne = $stmt->fetch(PDO::FETCH_OBJ);  
// Résultat : objet stdClass avec une propriété par colonne  
$ligne->id           = 1  
$ligne->pseudo       = 'Gandalf'  
$ligne->motDePasse   = 'Maia'  
$ligne->statutAdmin = 1  
  
echo $ligne->statutAdmin; // -> 1 (notation objet : $obj->propriete)
```

- **(+)** Notation orientée objet. Cohérent si on travaille avec des classes PHP.
- **(-)** Moins intuitif pour les débutants. FETCH_ASSOC souvent préféré.

Remarque – curseur et multiples fetch()

fetch() avance le curseur interne à chaque appel.

Pour tester plusieurs modes, relancer query() avant chaque test

ou commenter les blocs précédents (un seul mode actif à la fois).

6 – Sécurité : les injections SQL

6.1 Définition et mécanisme

Définition : injection SQL

L'injection SQL est une technique d'attaque qui consiste à insérer du code SQL malveillant dans les données envoyées par un utilisateur.

Si le programme construit sa requête par concaténation directe de variables, le code malveillant devient une partie de la requête et est exécuté par MySQL.

Le problème : fusion du SQL et des données

```
$pseudo = $_POST['pseudo']; // Valeur contrôlée par l'utilisateur
$sql = "SELECT * FROM utilisateurs WHERE pseudo = '$pseudo'";
//          ^^^^^^^
// PHP insère directement le texte de $pseudo dans le SQL
// Si $pseudo contient une apostrophe ou du SQL -> il s'exécute !
```

6.2 Les 3 types d'attaque

Type d'attaque	Valeur injectée	Effet
Erreur de syntaxe	Charles d'Artigue	Apostrophe -> erreur SQL
Bypass d'authentification	admin' OR '1'='1	Condition toujours vraie -> accès sans mdp
Destruction de données	'; DROP TABLE users; --	SQL arbitraire -> table détruite

Exemple d'injection SQL - bypass authentification

```
$pseudo = "Gandalf' OR '1'='1";
$sql = "SELECT * FROM utilisateurs WHERE pseudo='$pseudo'";
// La requête SQL formée devient :
// WHERE pseudo='Gandalf' OR '1'='1'
// La condition '1'='1' est TOUJOURS vraie !
// -> Tous les utilisateurs retournés (accès non autorisé)
```

6.3 Première protection : PDO::quote()

PDO::quote(string \$str) : string

PDO::quote() effectue deux opérations sur la chaîne passée en argument :

1. Elle échappe les caractères dangereux (apostrophes, antislashes...)
2. Elle entoure la valeur de guillemets simples

Remarque : quote() retourne la valeur AVEC ses guillemets.

Ne pas en ajouter dans la requête SQL.

Avant / Après PDO::quote()

```
$pseudo = "Gandalf' OR '1'='1";
$safe    = $pdo->quote($pseudo);
// Avant : Gandalf' OR '1'='1
// Après  : 'Gandalf\'' OR \'1\'=\'1\'
//          ^ guillemets ajoutés   ^ apostrophes échappées

// IMPORTANT : ne PAS ajouter de guillemets dans la requête
$sql = "SELECT * FROM utilisateurs WHERE pseudo = $safe";
```

- **Limite 1** : il faut appeler quote() pour CHAQUE variable. L'oubli d'un seul = vulnérabilité.
- **Limite 2** : laborieux avec de nombreux paramètres.
- **Solution recommandée** : les requêtes préparées (Ch. 8) gèrent l'échappement automatiquement.

7 – Passage de paramètres HTTP

7.1 Pourquoi passer des paramètres ?

HTTP est un protocole SANS ETAT (stateless)

Chaque requête HTTP est indépendante. Le serveur ne sait pas qui vous êtes d'une page à l'autre.

Pour transmettre une information entre deux pages PHP, il faut utiliser

l'un des 4 mécanismes de passage de paramètres : GET, POST, SESSION, COOKIE.

7.2 Comparatif des 4 méthodes

Variable	Stockage	Persistance / Sécurité / Usage
\$_GET	URL du navigateur	Non Faible Navigation, liens, filtres
\$_POST	Corps HTTP	Non Moyenne Formulaires, soumission
\$_SESSION	Serveur PHP	Oui (session) Bonne Auth, panier, données temp.
\$_COOKIE	Navigateur client	Oui (définie) Faible Préférences, 'Se souvenir'

7.3 GET et POST en détail

- **GET** : les paramètres sont dans l'URL après '?'. Format : page.php?cle=valeur&cle2=val2
Toujours encoder avec urlencode().
- **POST** : les paramètres sont dans le corps HTTP, invisibles dans l'URL. Nécessite un formulaire <form method="post">.

Fonction PHP	Rôle
urlencode(\$str)	Encode une chaîne pour l'URL. Espaces -> '+', spéciaux -> '%XX'.
htmlspecialchars(\$str)	Convertit les chars HTML en entités. Utiliser TOUJOURS pour afficher des données reçues.
isset(\$_GET['cle'])	Vérifie qu'une clé existe dans le tableau. Evite les erreurs 'undefined index'.

7.4 SESSION et COOKIE : persistance entre pages

SESSION : stockage côté serveur

session_start() doit être appelé en PREMIERE ligne de chaque page qui utilise SESSION.

\$_SESSION['cle'] = valeur -> accessible entre toutes les pages de la session.

Les données ne transitent JAMAIS chez le client (sécurité).

Usage : authentification, panier, flash messages.

COOKIE : stockage côté navigateur

Crée avec : setcookie('nom', 'valeur', time()+3600) -> expire dans 1h.

Lu avec \$_COOKIE['nom'] disponible à la PROCHAINE requête (pas la même page).

Visible et modifiable par l'utilisateur -> ne jamais stocker de données sensibles.

Usage : 'Se souvenir de moi', langue, thème.

8 – Requetés préparées

8.1 Pourquoi les requêtes préparées ?

Définition : requête préparée

Une requête préparée est une requête SQL dont la structure est envoyée à MySQL AVANT les valeurs.

MySQL compile et optimise la requête une première fois.

Ensuite, les valeurs sont envoyées séparément, comme données pures.

MySQL ne peut pas confondre les données avec des instructions SQL.

- **Avantage 1 – Sécurité** : SQL et données sont envoyés séparément. L'injection SQL est structurellement impossible. Même si une valeur contient 'OR 1=1', MySQL la traite comme une chaîne, jamais comme du SQL.
- **Avantage 2 – Performance** : MySQL compile la requête UNE SEULE FOIS lors de `prepare()`. Si la même requête est exécutée 100 fois avec des valeurs différentes, MySQL réutilise la version compilée.

Les 3 étapes d'une requête préparée :

1. **prepare()** : envoyer le squelette SQL avec des placeholders (? ou :nom). MySQL compile.
2. **MySQL compile** : analyse et optimise la requête. Les placeholders sont notés comme emplacements de données.
3. **execute()** : PHP envoie les valeurs réelles. MySQL les insère dans les emplacements.

8.2 Les placeholders : ? et :nom

Qu'est-ce qu'un placeholder ?

Un placeholder marque l'endroit dans le SQL où une valeur sera insérée de façon sécurisée.

Il existe deux syntaxes : positionnelle (?) et nommée (:nom).

PDO remplace chaque placeholder par la valeur correspondante après l'avoir échappée.

Syntaxe 1 : Placeholders positionnels (?)

Syntaxe avec ?

```
$stmt = $pdo->prepare(
    "SELECT * FROM utilisateurs WHERE pseudo = ?"
);
// execute() reçoit un tableau simple
// L'ordre correspond à l'ordre des ? dans la requête
$stmt->execute(['Gandalf']);

// Avec 2 paramètres :
$stmt = $pdo->prepare(
    "SELECT * FROM utilisateurs WHERE pseudo = ? AND motDePasse = ?"
);
$stmt->execute(['Aragorn', 'Dunedain']); // ordre OBLIGATOIRE
```

- **Tableau simple** : ['valeur1', 'valeur2']
- **Ordre obligatoire** : le 1er ? récite le 1er élément du tableau.
- **Usage** : pour 1 ou 2 paramètres.

Syntaxe 2 : Placeholders nommes (:nom)

Syntaxe avec :nom

```
$stmt = $pdo->prepare(
    "SELECT * FROM utilisateurs WHERE pseudo = :pseudo AND motDePasse = :mdp"
);
// execute() reçoit un tableau associatif
// L'ordre des clés n'a PAS d'importance
$stmt->execute([':pseudo' => 'Aragorn', ':mdp' => 'Dunedain']);
```

- **Tableau associatif** : [':nom' => valeur]
- **Ordre libre** : la clé identifie le placeholder.
- **Usage recommande** : pour 3 paramètres ou plus, ou pour plus de lisibilité.

Attention

Ne jamais mélanger les deux syntaxes (? et :nom) dans la même requête.

Choisir l'une ou l'autre.

8.3 bindParam() vs bindValue()

Deux méthodes pour lier des paramètres a un placeholder

Ces deux méthodes lient un paramètre a un placeholder AVANT execute().

La différence fondamentale est dans QUAND la valeur est lue.

bindParam() : lie par REFERENCE (adresse memoire)

PDOStatement::bindParam(string \$param, mixed &\$var)

```
$stmt->bindParam(':pseudo', $pseudo);
// bindParam() enregistre l'ADRESSE MEMOIRE de $pseudo
// La valeur est lue AU MOMENT de execute(), pas maintenant

$pseudo = 'Gandalf';
$stmt->bindParam(':pseudo', $pseudo);
$pseudo = 'Aragorn'; // On change la variable
$stmt->execute();
// execute() lit la valeur ACTUELLE -> 'Aragorn' (pas 'Gandalf' !)
```

- **Usage idéal** : boucles répétées. prepare() 1 fois, bindParam() 1 fois, execute() dans la boucle.
- **Analogie** : étiquette sur une boîte. Si la boîte change de contenu, l'étiquette pointe toujours vers elle.

bindValue() : lie par VALEUR (copie immediate)

PDOStatement::bindValue(string \$param, mixed \$val)

```
$stmt->bindValue(':pseudo', $pseudo);
// bindValue() copie la valeur au moment de l'appel (comme une photo)

$pseudo = 'Gandalf';
$stmt->bindValue(':pseudo', $pseudo);
$pseudo = 'Aragorn'; // On change APRES la photo
$stmt->execute();
```

```
// execute() utilise la valeur capturée -> 'Gandalf' (pas 'Aragorn' !)
```

```
// bindValue permet aussi les valeurs littérales directes :
```

```
$stmt->bindValue(':pseudo', 'Legolas'); // pas de variable nécessaire
```

- **Usage idéal** : valeur fixe ou littérale.
- **Analogie** : photocopie du contenu. Si l'original change après, la copie reste identique.

Critere	bindParam()	bindValue()
Lie par	Reference (adresse memoire)	Valeur (copie)
Lit la valeur a	execute()	bindParam/Value()
Changer \$var apres	Effet : lit la nouvelle valeur	Aucun effet
Valeur litterale	Non possible	Oui possible
Usage ideal	Boucles répètes	Valeur fixe/littérale
Echappement auto	Oui	Oui

Synthèse – Les 8 concepts fondamentaux de PDO

Concept	Définition essentielle
1. PDO	Couche d'abstraction PHP. Même syntaxe pour MySQL, SQLite, PostgreSQL...
2. DSN	Chaine de connexion : 'driver:host=...;dbname=...;charset=...'
3. Options PDO	ERRMODE_EXCEPTION + FETCH_ASSOC + EMULATE_PREPARES=false. Toujours configurer.
4. query() + fetch()	Pour les requêtes SQL FIXES sans variable externe. Curseur avance à chaque fetch().
5. Modes fetch	FETCH_NUM (indices), FETCH_ASSOC (noms colonnes, recommande), FETCH_OBJ (objet).
6. Injection SQL	Danger de la concaténation directe. Apostrophes dans les données = SQL exécuté.
7. GET/POST/SESSION/COOKIE	4 méthodes de passage de paramètres entre pages. Chacune a son usage propre.
8. requêtes préparées	prepare() + execute(). SQL et données séparees. Protection absolue + performance.